# QuickstartÂ

Eager to get started? This page gives a good introduction to Flask. It assumes you already have Flask installed. If you do not, head over to the Installation section.

# A Minimal ApplicationÂ

A minimal Flask application looks something like this:

# A Minimal ApplicationÂ

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'
```

# A Minimal ApplicationÂ

So what did that code do?

# A Minimal ApplicationÂ

Just save it as hello.py or something similar. Make sure to not call your application flask.py because this would conflict with Flask itself.

# A Minimal ApplicationÂ

To run the application you can either use the flask command or pythonâ  s -m switch with Flask. Before you can do that you need to tell your terminal the application to work with by exporting the FLASK_APP environment variable:

# A Minimal ApplicationÂ

```
$ export FLASK_APP=hello.py
$ flask run
 * Running on http://127.0.0.1:5000/
```

# A Minimal ApplicationÂ

If you are on Windows you need to use set instead of export.

# A Minimal ApplicationÂ

Alternatively you can use python -m flask:

# A Minimal ApplicationÂ

```
$ export FLASK_APP=hello.py
$ python -m flask run
 * Running on http://127.0.0.1:5000/
```

# A Minimal ApplicationÂ

This launches a very simple builtin server, which is good enough for testing but probably not what you want to use in production. For deployment options see Deployment Options.

# A Minimal ApplicationÂ

Now head over to http://127.0.0.1:5000/, and you should see your hello world greeting.

# A Minimal ApplicationÂ

Externally Visible Server

# A Minimal ApplicationÂ

If you run the server you will notice that the server is only accessible from your own computer, not from any other in the network. This is the default because in debugging mode a user of the application can execute arbitrary Python code on your computer.

# A Minimal ApplicationÂ

If you have the debugger disabled or trust the users on your network, you can make the server publicly available simply by adding –host=0.0.0.0 to the command line:

# A Minimal ApplicationÂ

```
flask run --host=0.0.0.0
```

# A Minimal ApplicationÂ

This tells your operating system to listen on all public IPs.

# What to do if the Server does not StartÂ

In case the python -m flask fails or flask does not exist, there are multiple reasons this might be the case. First of all you need to look at the error message.

# What to do if the Server does not StartÂ

Versions of Flask older than 0.11 use to have different ways to start the application. In short, the flask command did not exist, and neither did python -m flask. In that case you have two options: either upgrade to newer Flask versions or have a look at the Development Server docs to see the alternative method for running a server.

# What to do if the Server does not StartÂ

The FLASK_APP environment variable is the name of the module to import at flask run. In case that module is incorrectly named you will get an import error upon start (or if debug is enabled when you navigate to the application). It will tell you what it tried to import and why it failed.

# What to do if the Server does not StartÂ

The most common reason is a typo or because you did not actually create an app object.

# Debug ModeÂ

(Want to just log errors and stack traces? See Application Errors)

# Debug ModeÂ

The flask script is nice to start a local development server, but you would have to restart it manually after each change to your code. That is not very nice and Flask can do better. If you enable debug support the server will reload itself on code changes, and it will also provide you with a helpful debugger if things go wrong.

# Debug ModeÂ

To enable debug mode you can export the FLASK_DEBUG environment variable before running the server:

# Debug ModeÂ

```
$ export FLASK_DEBUG=1
$ flask run
```

# Debug ModeÂ

(On Windows you need to use set instead of export).

# Debug ModeÂ

This does the following things:

# Debug ModeÂ

There are more parameters that are explained in the Development Server docs.

# Debug ModeÂ

Attention

# Debug ModeÂ

Even though the interactive debugger does not work in forking environments (which makes it nearly impossible to use on production servers), it still allows the execution of arbitrary code. This makes it a major security risk and therefore it must never be used on production machines.

# Debug ModeÂ

Screenshot of the debugger in action:

# Debug ModeÂ

Have another debugger in mind? See Working with Debuggers.

# RoutingÂ

Modern web applications have beautiful URLs. This helps people remember the URLs, which is especially handy for applications that are used from mobile devices with slower network connections. If the user can directly go to the desired page without having to hit the index page it is more likely they will like the page and come back next time.

# RoutingÂ

As you have seen above, the route() decorator is used to bind a function to a URL. Here are some basic examples:

# RoutingÂ

```
@app.route('/')
def index():
    return 'Index Page'

@app.route('/hello')
def hello():
    return 'Hello, World'
```

# RoutingÂ

But there is more to it! You can make certain parts of the URL dynamic and attach multiple rules to a function.

# RoutingÂ

To add variable parts to a URL you can mark these special sections as <variable_name>. Such a part is then passed as a keyword argument to your function. Optionally a converter can be used by specifying a rule with <converter:variable_name>. Here are some nice examples:

# RoutingÂ

```
@app.route('/user/<username>')
def show_user_profile(username):
    # show the user profile for that user
    return 'User %s' % username

@app.route('/post/<int:post_id>')
def show_post(post_id):
   # show the post with the given id, the id is an integer
    return 'Post %d' % post_id
```

# RoutingÂ

The following converters exist:

# RoutingÂ

Unique URLs / Redirection Behavior

# RoutingÂ

Flaskâ  s URL rules are based on Werkzeugâ  s routing module. The idea behind that module is to ensure beautiful and unique URLs based on precedents laid down by Apache and earlier HTTP servers.

# RoutingÂ

Take these two rules:

# RoutingÂ

```
@app.route('/projects/')
def projects():
    return 'The project page'

@app.route('/about')
def about():
    return 'The about page'
```

# RoutingÂ

Though they look rather similar, they differ in their use of the trailing slash in the URL definition. In the first case, the canonical URL for the projects endpoint has a trailing slash. In that sense, it is similar to a folder on a filesystem. Accessing it without a trailing slash will cause Flask to redirect to the canonical URL with the trailing slash.

# RoutingÂ

In the second case, however, the URL is defined without a trailing slash, rather like the pathname of a file on UNIX-like systems. Accessing the URL with a trailing slash will produce a 404 â Not Foundâ error.

# RoutingÂ

This behavior allows relative URLs to continue working even if the trailing slash is omitted, consistent with how Apache and other servers work. Also, the URLs will stay unique, which helps search engines avoid indexing the same page twice.

# RoutingÂ

If it can match URLs, can Flask also generate them? Of course it can. To build a URL to a specific function you can use the url_for() function. It accepts the name of the function as first argument and a number of keyword arguments, each corresponding to the variable part of the URL rule. Unknown variable parts are appended to the URL as query parameters. Here are some examples:

# RoutingÂ

```
>>> from flask import Flask, url_for
>>> app = Flask(__name__)
>>> @app.route('/')
... def index(): pass
...
>>> @app.route('/login')
... def login(): pass
...
>>> @app.route('/user/<username>')
... def profile(username): pass
...
>>> with app.test_request_context():
...  print url_for('index')
...  print url_for('login')
...  print url_for('login', next='/')
...  print url_for('profile', username='John Doe')
```

# RoutingÂ

(This also uses the test_request_context() method, explained below. It tells Flask to behave as though it is handling a request, even though we are interacting with it through a Python shell. Have a look at the explanation below. Context Locals).

# RoutingÂ

Why would you want to build URLs using the URL reversing function url_for() instead of hard-coding them into your templates? There are three good reasons for this:

# RoutingÂ

HTTP (the protocol web applications are speaking) knows different methods for accessing URLs. By default, a route only answers to GET requests, but that can be changed by providing the methods argument to the route() decorator. Here are some examples:

# RoutingÂ

```
from flask import request

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        do_the_login()
    else:
        show_the_login_form()
```

# RoutingÂ

If GET is present, HEAD will be added automatically for you. You donâ t have to deal with that. It will also make sure that HEAD requests are handled as the HTTP RFC (the document describing the HTTP protocol) demands, so you can completely ignore that part of the HTTP specification. Likewise, as of Flask 0.6, OPTIONS is implemented for you automatically as well.

# RoutingÂ

You have no idea what an HTTP method is? Worry not, here is a quick introduction to HTTP methods and why they matter:

# RoutingÂ

The HTTP method (also often called â the verbâ ) tells the server what the client wants to do with the requested page. The following methods are very common:

# RoutingÂ

Now the interesting part is that in HTML4 and XHTML1, the only methods a form can submit to the server are GET and POST. But with JavaScript and future HTML standards you can use the other methods as well. Furthermore HTTP has become quite popular lately and browsers are no longer the only clients that are using HTTP. For instance, many revision control systems use it.

# Static FilesÂ

Dynamic web applications also need static files. Thatâ s usually where the CSS and JavaScript files are coming from. Ideally your web server is configured to serve them for you, but during development Flask can do that as well. Just create a folder called static in your package or next to your module and it will be available at /static on the application.

# Static FilesÂ

To generate URLs for static files, use the special 'static' endpoint name:

# Static FilesÂ

```
url_for('static', filename='style.css')
```

# Static FilesÂ

The file has to be stored on the filesystem as static/style.css.

# Rendering TemplatesÂ

Generating HTML from within Python is not fun, and actually pretty cumbersome because you have to do the HTML escaping on your own to keep the application secure. Because of that Flask configures the Jinja2 template engine for you automatically.

# Rendering TemplatesÂ

To render a template you can use the render_template() method. All you have to do is provide the name of the template and the variables you want to pass to the template engine as keyword arguments. Hereâ  s a simple example of how to render a template:

# Rendering TemplatesÂ

```
from flask import render_template

@app.route('/hello/')
@app.route('/hello/<name>')
def hello(name=None):
    return render_template('hello.html', name=name)
```

# Rendering TemplatesÂ

Flask will look for templates in the templates folder. So if your application is a module, this folder is next to that module, if itâ s a package itâ s actually inside your package:

# Rendering TemplatesÂ

Case 1: a module:

# Rendering TemplatesÂ

```
/application.py
/templates
    /hello.html
```

# Rendering TemplatesÂ

Case 2: a package:

# Rendering TemplatesÂ

```
/application
    /__init__.py
    /templates
        /hello.html
```

# Rendering TemplatesÂ

For templates you can use the full power of Jinja2 templates. Head over to the official Jinja2 Template Documentation for more information.

# Rendering TemplatesÂ

Here is an example template:

# Rendering TemplatesÂ

```
<!doctype html>
<title>Hello from Flask</title>
{% if name %}
  <h1>Hello {{ name }}!</h1>
{% else %}
  <h1>Hello, World!</h1>
{% endif %}
```

# Rendering TemplatesÂ

Inside templates you also have access to the request, session and g [1] objects as well as the get_flashed_messages() function.

# Rendering TemplatesÂ

Templates are especially useful if inheritance is used. If you want to know how that works, head over to the Template Inheritance pattern documentation. Basically template inheritance makes it possible to keep certain elements on each page (like header, navigation and footer).

# Rendering TemplatesÂ

Automatic escaping is enabled, so if name contains HTML it will be escaped automatically. If you can trust a variable and you know that it will be safe HTML (for example because it came from a module that converts wiki markup to HTML) you can mark it as safe by using the Markup class or by using the |safe filter in the template. Head over to the Jinja 2 documentation for more examples.

# Rendering TemplatesÂ

Here is a basic introduction to how the Markup class works:

# Rendering TemplatesÂ

```
>>> from flask import Markup
>>> Markup('<strong>Hello %s!</strong>') % '<blink>hacker</b
Markup(u'<strong>Hello &lt;blink&gt;hacker&lt;/blink&gt;!</
>>> Markup.escape('<blink>hacker</blink>')
Markup(u'&lt;blink&gt;hacker&lt;/blink&gt;')
>>> Markup('<em>Marked up</em> &raquo; HTML').striptags()
u'Marked up \xbb HTML'
```

# Rendering TemplatesÂ

Changed in version 0.5: Autoescaping is no longer enabled for all templates. The following extensions for templates trigger autoescaping: .html, .htm, .xml, .xhtml. Templates loaded from a string will have autoescaping disabled.

# Accessing Request DataÂ

For web applications itâ s crucial to react to the data a client sends to the server. In Flask this information is provided by the global request object. If you have some experience with Python you might be wondering how that object can be global and how Flask manages to still be threadsafe. The answer is context locals:

# Accessing Request DataÂ

## Insider Information

# Accessing Request DataÂ

If you want to understand how that works and how you can implement tests with context locals, read this section, otherwise just skip it.

# Accessing Request DataÂ

Certain objects in Flask are global objects, but not of the usual kind. These objects are actually proxies to objects that are local to a specific context. What a mouthful. But that is actually quite easy to understand.

# Accessing Request DataÂ

Imagine the context being the handling thread. A request comes in and the web server decides to spawn a new thread (or something else, the underlying object is capable of dealing with concurrency systems other than threads). When Flask starts its internal request handling it figures out that the current thread is the active context and binds the current application and the WSGI environments to that context (thread). It does that in an intelligent way so that one application can invoke another application without breaking.

# Accessing Request DataÂ

So what does this mean to you? Basically you can completely ignore that this is the case unless you are doing something like unit testing. You will notice that code which depends on a request object will suddenly break because there is no request object. The solution is creating a request object yourself and binding it to the context. The easiest solution for unit testing is to use the test_request_context() context manager. In combination with the with statement it will bind a test request so that you can interact with it. Here is an example:

# Accessing Request DataÂ

```
from flask import request

with app.test_request_context('/hello', method='POST'):
    # now you can do something with the request until the
     # end of the with block, such as basic assertions:
     assert request.path == '/hello'
     assert request.method == 'POST'
```

# Accessing Request DataÂ

The other possibility is passing a whole WSGI environment to the request_context() method:

# Accessing Request DataÂ

```
from flask import request

with app.request_context(environ):
    assert request.method == 'POST'
```

# Accessing Request DataÂ

The request object is documented in the API section and we will not cover it here in detail (see request). Here is a broad overview of some of the most common operations. First of all you have to import it from the flask module:

# Accessing Request DataÂ

```
from flask import request
```

# Accessing Request DataÂ

The current request method is available by using the method attribute. To access form data (data transmitted in a POST or PUT request) you can use the form attribute. Here is a full example of the two attributes mentioned above:

# Accessing Request DataÂ

```
@app.route('/login', methods=['POST', 'GET'])
def login():
    error = None
    if request.method == 'POST':
        if valid_login(request.form['username'],
                       request.form['password']):
        return log_the_user_in(request.form['username'])
        else:
            error = 'Invalid username/password'
    # the code below is executed if the request method
    # was GET or the credentials were invalid
    return render_template('login.html', error=error)
```

# Accessing Request DataÂ

What happens if the key does not exist in the form attribute? In that case a special KeyError is raised. You can catch it like a standard KeyError but if you donâ  t do that, a HTTP 400 Bad Request error page is shown instead. So for many situations you donâ  t have to deal with that problem.

# Accessing Request DataÂ

To access parameters submitted in the URL (?key=value) you can use the args attribute:

# Accessing Request DataÂ

```
searchword = request.args.get('key', '')
```

# Accessing Request DataÂ

We recommend accessing URL parameters with get or by catching the KeyError because users might change the URL and presenting them a 400 bad request page in that case is not user friendly.

# Accessing Request DataÂ

For a full list of methods and attributes of the request object, head over to the request documentation.

# Accessing Request DataÂ

You can handle uploaded files with Flask easily. Just make sure not to forget to set the enctype=”multipart/form-data” attribute on your HTML form, otherwise the browser will not transmit your files at all.

# Accessing Request DataÂ

Uploaded files are stored in memory or at a temporary location on the filesystem. You can access those files by looking at the files attribute on the request object. Each uploaded file is stored in that dictionary. It behaves just like a standard Python file object, but it also has a save() method that allows you to store that file on the filesystem of the server. Here is a simple example showing how that works:

# Accessing Request DataÂ

```
from flask import request

@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        f = request.files['the_file']
        f.save('/var/www/uploads/uploaded_file.txt')
    ...
```

# Accessing Request DataÂ

If you want to know how the file was named on the client before it was uploaded to your application, you can access the filename attribute. However please keep in mind that this value can be forged so never ever trust that value. If you want to use the filename of the client to store the file on the server, pass it through the secure_filename() function that Werkzeug provides for you:

# Accessing Request DataÂ

```
from flask import request
from werkzeug.utils import secure_filename

@app.route('/upload', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        f = request.files['the_file']
     f.save('/var/www/uploads/' + secure_filename(f.filenam
    ...
```

# Accessing Request DataÂ

For some better examples, checkout the Uploading Files pattern.

# Accessing Request DataÂ

To access cookies you can use the cookies attribute. To set cookies you can use the set_cookie method of response objects. The cookies attribute of request objects is a dictionary with all the cookies the client transmits. If you want to use sessions, do not use the cookies directly but instead use the Sessions in Flask that add some security on top of cookies for you.

# Accessing Request DataÂ

Reading cookies:

# Accessing Request DataÂ

```
from flask import request

@app.route('/')
def index():
    username = request.cookies.get('username')
   # use cookies.get(key) instead of cookies[key] to not get a
    # KeyError if the cookie is missing.
```

# Accessing Request DataÂ

Storing cookies:

# Accessing Request DataÂ

```
from flask import make_response

@app.route('/')
def index():
    resp = make_response(render_template(...))
    resp.set_cookie('username', 'the username')
    return resp
```

# Accessing Request DataÂ

Note that cookies are set on response objects. Since you normally just return strings from the view functions Flask will convert them into response objects for you. If you explicitly want to do that you can use the make_response() function and then modify it.

# Accessing Request DataÂ

Sometimes you might want to set a cookie at a point where the response object does not exist yet. This is possible by utilizing the Deferred Request Callbacks pattern.

# Accessing Request DataÂ

For this also see About Responses.

# Redirects and ErrorsÂ

To redirect a user to another endpoint, use the redirect() function; to abort a request early with an error code, use the abort() function:

# Redirects and ErrorsÂ

```
from flask import abort, redirect, url_for

@app.route('/')
def index():
    return redirect(url_for('login'))

@app.route('/login')
def login():
    abort(401)
    this_is_never_executed()
```

# Redirects and ErrorsÂ

This is a rather pointless example because a user will be redirected from the index to a page they cannot access (401 means access denied) but it shows how that works.

# Redirects and ErrorsÂ

By default a black and white error page is shown for each error code. If you want to customize the error page, you can use the errorhandler() decorator:

# Redirects and ErrorsÂ

```
from flask import render_template

@app.errorhandler(404)
def page_not_found(error):
    return render_template('page_not_found.html'), 404
```

# Redirects and ErrorsÂ

Note the 404 after the render_template() call. This tells Flask that the status code of that page should be 404 which means not found. By default 200 is assumed which translates to: all went well.

# Redirects and ErrorsÂ

See Error handlers for more details.

# About ResponsesÂ

The return value from a view function is automatically converted into a response object for you. If the return value is a string itâ  s converted into a response object with the string as response body, a 200 OK status code and a text/html mimetype. The logic that Flask applies to converting return values into response objects is as follows:

# About ResponsesÂ

If you want to get hold of the resulting response object inside the view you can use the make_response() function.

# About ResponsesÂ

Imagine you have a view like this:

# About ResponsesÂ

```
@app.errorhandler(404)
def not_found(error):
    return render_template('error.html'), 404
```

# About ResponsesÂ

You just need to wrap the return expression with make_response() and get the response object to modify it, then return it:

# About ResponsesÂ

```
@app.errorhandler(404)
def not_found(error):
   resp = make_response(render_template('error.html'), 404)
    resp.headers['X-Something'] = 'A value'
    return resp
```

# SessionsÂ

In addition to the request object there is also a second object called session which allows you to store information specific to a user from one request to the next. This is implemented on top of cookies for you and signs the cookies cryptographically. What this means is that the user could look at the contents of your cookie but not modify it, unless they know the secret key used for signing.

# SessionsÂ

In order to use sessions you have to set a secret key. Here is how sessions work:

# SessionsÂ

```
from flask import Flask, session, redirect, url_for, escape,

app = Flask(__name__)

@app.route('/')
def index():
    if 'username' in session:
     return 'Logged in as %s' % escape(session['username'])
    return 'You are not logged in'

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        session['username'] = request.form['username']
        return redirect(url_for('index'))
    return '''
```

# SessionsÂ

The escape() mentioned here does escaping for you if you are not using the template engine (as in this example).

# SessionsÂ

How to generate good secret keys

# SessionsÂ

The problem with random is that itâ s hard to judge what is truly random. And a secret key should be as random as possible. Your operating system has ways to generate pretty random stuff based on a cryptographic random generator which can be used to get such a key:

# SessionsÂ

```
>>> import os
>>> os.urandom(24)
'\xfd{H\xe5<\x95\xf9\xe3\x96.5\xd1\x01O<!\xd5\xa2\xa0\x9fR'

Just take that thing and copy/paste it into your code and you'
```

# SessionsÂ

A note on cookie-based sessions: Flask will take the values you put into the session object and serialize them into a cookie. If you are finding some values do not persist across requests, cookies are indeed enabled, and you are not getting a clear error message, check the size of the cookie in your page responses compared to the size supported by web browsers.

# SessionsÂ

Besides the default client-side based sessions, if you want to handle sessions on the server-side instead, there are several Flask extensions that support this.

# Message FlashingÂ

Good applications and user interfaces are all about feedback. If the user does not get enough feedback they will probably end up hating the application. Flask provides a really simple way to give feedback to a user with the flashing system. The flashing system basically makes it possible to record a message at the end of a request and access it on the next (and only the next) request. This is usually combined with a layout template to expose the message.

# Message FlashingÂ

To flash a message use the flash() method, to get hold of the messages you can use get_flashed_messages() which is also available in the templates. Check out the Message Flashing for a full example.

# LoggingÂ

New in version 0.3.

# LoggingÂ

Sometimes you might be in a situation where you deal with data that should be correct, but actually is not. For example you may have some client-side code that sends an HTTP request to the server but itâ s obviously malformed. This might be caused by a user tampering with the data, or the client code failing. Most of the time itâ s okay to reply with 400 Bad Request in that situation, but sometimes that wonâ t do and the code has to continue working.

# LoggingÂ

You may still want to log that something fishy happened. This is where loggers come in handy. As of Flask 0.3 a logger is preconfigured for you to use.

# LoggingÂ

Here are some example log calls:

# LoggingÂ

```
app.logger.debug('A value for debugging')
app.logger.warning('A warning occurred (%d apples)', 42)
app.logger.error('An error occurred')
```

# LoggingÂ

The attached logger is a standard logging Logger, so head over to the official logging documentation for more information.

# LoggingÂ

Read more on Application Errors.

# Hooking in WSGI MiddlewaresÂ

If you want to add a WSGI middleware to your application you can wrap the internal WSGI application. For example if you want to use one of the middlewares from the Werkzeug package to work around bugs in lighttpd, you can do it like this:

# Hooking in WSGI MiddlewaresÂ

```
from werkzeug.contrib.fixers import LighttpdCGIRootFix
app.wsgi_app = LighttpdCGIRootFix(app.wsgi_app)
```

# Using Flask ExtensionsÂ

Extensions are packages that help you accomplish common tasks. For example, Flask-SQLAlchemy provides SQLAlchemy support that makes it simple and easy to use with Flask.

# Using Flask ExtensionsÂ

For more on Flask extensions, have a look at Flask Extensions.

# Deploying to a Web ServerÂ

Ready to deploy your new Flask app? Go to Deployment Options.